[illegible] is recursed.

18/9/20
Pg 4 Qy 2

Q. Why we need to design our own I/P fun
Q. In function calls we require in our
Q. Explain what getchar() do
Q. Why we write int ch not char ch
Q. Write full code to get custom I/P having our own I/P function.
Q. Putchar Prototype
Q. What Putchar function is
Q. Why int ch not char ch?

```
#include<stdio.h>
int main(){
    int ch;
    for(ch='A'; ch<='Z'; ch++)
    putchar(ch);
    return 0;
}
```

Scanned with CamScanner

Q. which didn't work properly?
a) printf("%c", '\n');
b) printf("%c", "\n");
c) putchar('\n');
d) putchar("\n");
e) puts('\n');
f) puts("\n");
g) printf("%s", '\n');
h) printf("%s", "\n");

Q. Which operations can be performed on string
Q. library header
Q. Syntax to copy a string
Prototype
Q. Where strcpy used
Q. what ,, returns
Q. Write a program to use strcpy in printf
Q. Prog to to chain together a series of strcpy calls
Q. Disadv of strcpy
Q. Behaviour when len str2 is greater than len of chain array of str1
Q. New function to solve
Q. Prototype of strncpy
Q. code of ,,
Q. In strncpy what is disadv
1. if size of str1, str2 same
overcome this disadv code
function to get len of any
Prototype
size of full explan

Scanned with CamScanner

Page 05
Date

Q. why we need character
Q. strlen & NULL character
Q. Eg code
Q. String concatenation function
Q. Prototype strcat
Q. What strcat returns
Q. Eg code
Q. Caution - when undefined behaviour
Q. Eg
Q. Better version of strcat
Q. strncat & NULL
Q. Eg code
Q. strcmp prototype
Q. What strcmp returns
Q. In ASCII Character set - 3
Q. strcmp, cmpclen - 3 thing
Q. - 2nd
Q. eg code
Q. eg code
Q. Array of strings syntax
Q. Visualize "Zorger", "Apple", "Banana", "Pineapple"
Q. Drawback
Q. Alternative
Q. Visualize
Q. char p[20];
char *s = "string";
int length = strlen(s);
p[i] = s[length-i];
printf("%s", p);

OP a) gnirts b) gnirt c) string d) None

Scanned with CamScanner

Gate 2011

Q) char c[] = "GATE2011"
char *p = c;
printf("%s", p + p[3] - p[1]);
a) GATE2011 b) E2011 c) 2011 d) 011

Gate 2015

Q- Consider the following function written in C. The o/p of function on I/P "ABCD" is:

```
Void foo(char *a){
if(*a && *a != '\0'){
foo(a+1);
putchar(*a);
}}
```

a) ABCD EFGH
b) ABCD
c) HGFEDCBA
d) DCBA

Q- Void fun1(char *s1, char *s2){
char *tmp;
tmp = s1;
s1 = s2;
s2 = tmp;}
Void fun2(char **s1, char **s2){
char *tmp;
tmp = *s1;
*s1 = *s2;
*s2 = tmp;}
int main(){
char *str1 = "Hi", *str2 = "Bye";
fun1(str1, str2); printf("%s %s", str1, str2);
fun2(&str1, &str2); printf("%s %s", str1, str2);
}

Scanned with CamScanner

Gate 2017

O/P of

```
#include<stdio.h>
#include<string.h>
int main(){
  char *c = "GATE CSIT 2017";
  char *p = c;
  printf("%d", (int)strlen(c+2[p]-6[p+1]));
}
```

a) 1 c) 4
b) 2 d) 6

Q: Explain type casting from A to B?
Q: Function Pointer - defn
Q: How to declare a pointer to an array.
Q: int *ptr[10];
Q: Why wrong & what's correct?
Q: How to declare a pointer to function?
Q: Declare pointer for
int add(int a, int b){
return (a+b); }
— 3 ways
Q: Automatic
Q: Application of function pointers
Q: Function → not runtime
Q: We want to design a calculator-operations
Q: Code of calculator using method
Q: Structure — Problem statement of car & garage
Q: 1st way to solve
Q: 2nd way -
Q: Why not recommended
Q: Solar
Q: What is structure

Scanned with CamScanner

Page
Date

Syntax & variables Structure
Code

Lecture #2

Global struct

Can we create it in local space
eg code

Disadv of this code

How to solve eg code

Syntax of typedef

What is typedef used for

Eg code typedef

Structure declaration typedef separate

use of typedef in structure
eg code

Initializing struct variables - wrong way
correct

How to access members of structure
eg code

What is designated initialization
order
eg code

What is array of structure
why?
eg code

Accessing members of structure using structure pointer
eg code

ptr -> x is similar

How memory allocated for structure members

Scanned with CamScanner

Page 110
Date

Q) Name of 2 concepts to allot memory
Q) What amount of memory processor can allocate
Q) 32 bits processor
Q) 64 " "
Q) Draw CPU cycles for char a, char b, int c.
Q) Explain wasted CPU cycles
Q) How # of cycles saved
Q) Draw & find size of

```
struct abc{
    char a;
    char b;
    int c;
};
```

in 32-bit systems

o/p if char a; int b; char c;

17/9/20 Pg-30

Q) Disadv of structure padding
Q) 2nd concept of memory allocation
Q) Changes in code
Q) What #pragma used for
Q) Use structure packing & tell size of

```
struct abc{
    char a;
    int b;
    char c;
}var;
```

Q) Imp point of trade off

Q) #include<stdio.h>

```
struct Ournode{
    char x, y, z;
};
int main(){
    struct Ournode k = {'a', 'b', 'c'};
    struct Ournode *kp = &k;
    printf("%c, %c", *(char *)kp+1, *(char *)kp+2);
}
```

a) a, c
b) a, a+2
c) b, a+2
d) b, c

Scanned with CamScanner

Gate 2000

Page
Date

Q Following C declarations define s to be.

```
struct node {
    int i;
    float j;
} *s[10];
```

a) An array, each element of which is a pointer to a structure of type node.
b) A structure of 2 fields, each field being a pointer to an array of 10 elements
c) A structure of 3 fields, an integer, a float and an array of 10 elements
d) An array, each element of which is a structure of type node.

Q- What is union
Q- Diff from structure how?
Q- Eg code
Q- Utilize
Q- Fast, Eg code
Q- How size of union decided

```
union abc {
    int a;
    char b;
    double c;
    float d;
};
int main() {
printf("%ld", sizeof(union abc));
}
```

Q- How we can access member of union using pointers
Q- Eg Code

Q- short = 2 bytes find memory for variable t
long = 8 bytes a) 22 bytes b) 14 bytes c) 8 bytes d) 10 bytes
float = 4 bytes. struct{short s[5]; union{float y; long z;} u;} t;

Scanned with CamScanner

Application of union & Assumption of structure
Initial code with structure
Memory
Using Unions
Memory
Application of union (part-2)
design an array of heterogenous data
why not structures)
eg figures
Enumeration in C
what it do?
eg of it
what enum returns
Default values
How it diff from #define -2
eg for 1
Part #1 of enum
eg code
Part #2 of enum
eg code
value of t in enum point{y=2,x=34,t,3=0};
Part #3 of enum
eg error code
Part #4
eg code

```
int main(){
enum point1 = {x=34, y=2, z=0};
enum point2 = {x=4, p=25, q=14};
printf("%d %d %d %d %d", x, y, z, p, q);
```

Scanned with CamScanner

Page
Date

Q. The following structures are designed to store information about objects on a graphics screen.

struct point {int x, y;}
struct rectangle {struct point upper left, lower right;}

A point structure stores the x & y coordinates of a point. Store the coordinates of upper left & lower right corners of the rectangle.

Q.
```
struct {
double a;
union {
  char b[4];
  double c;
  int d;
} e;
char f[4];
} s;
```

```
union {
double a;
struct {
  char b[4];
  double c;
  int d;
} e;
char f[4];
} u;
```

Memory for both if compiler leaves no holes

Q. Dictionary def$^n$ of data
Eg
When data becomes info — Eg
Info def$^n$
Why data structures?
What is a data structure?
Eg
Compare array & variables
Storing string & char variables
Real life Eg #1
Visualization

Page
Date

- Which DS is used to store image
  - How
- Storing friendship info on a social N/w site?
  - Diag
- What is a data type - 2
  - eg int type - 2
  - eg float " - 2
    - operations not allowed
- User defined data type - 2
  - Eqs - 3
  - eg of structure
  - ADT expand
    - ADT defn
  - eg "
- 5 operations on stack
  - only —
- What we can think of ADT as
- ADT can be implemented in one way/multiple ways
  - eg "
- 2 types of program
- Importance of ADT through client & Implementation program
- 2 advantages
- Conclusion
- What is a data structure
  - In other words, " "
  - eg
- ADT tells us — DS tells us —
- How to know which DS to use for ADT
- 3 advantages of DS
- 2 types of DS
- Linear DS - defn 1
  - defn 2
- eg

Scanned with CamScanner

Non linear - 2 defn
2 eg
Visualise
Event further classification - 2
Static DS - defn
Advantage
Disadvantage
Eg
Dynamic DS - defn
Adv
Disad
Eg
How n of Asympto DS measured
on what basis we would compare the time complexity of the DS?
Elaborate # of operations except by adding data at beginning of
array
linked list
Conclusion
How to find time complexity of operation - Method
Problems of method 1 - 2
Points to be noted - 1
" " 1 - 2
Eg of hello world
How compared using f(n)?
How to find f(n)?
We are interested in — not
Eg of $6n^2 + 6n + 2$
Graph & table
Short of
Define Asymptotic Analysis

Scanned with CamScanner

Big O- Notation

Defn

Simple meaning:

f(n)=n  g(n)=2n  find

f(n)=4n+3  g(n)=3n → Full elaborate

How we known time complexity from big-O notation

We are interested in...

What we check for

Why we check

Eg

Issues

Solution

Best part of Big O

What Dont upper bound tells

f(n)=5n²+4  g(n)=n²

Big-O elimination in this eg

Growth rate of 6 standard functions (5 n²)

Calculate f(n) for program to find sum of first n natural numbers

find suitable g(n) for it

So we have easy way

Code

g(n)

Guidelines for asymptotic analysis loops nested loops consecutive statements if-else

Rigs of all

Logarithmic complexity. Expand log(n)

Page
Date

Q for loop eg for log time complexity
i) When & how log time achieved
ii) Derive for
```
for(i=1; i<k;)
    i=i*2;
```

Q. Derive for
```
for(i=n; i>=1;){
    i=i/2;
}
```
Derive y

Q Find time complexity of
```
void fun(int n){
    int i, j, k, count=0;
    for(i=n/2; i<=n; i++)
        for(j=1; j+n/2<=n; j++)
            for(k=1; k<=n; k=k*2)
                count++;
}
```

Q Time complexity of
```
void fun(int n){
    if(n<=1) return;
    int i, j;
    for(i=1; i<=n; i++)
        for(j=1; j<=n; j++){
            printf("Hello\n");
            break;
        }
}
```

Or
```
void fun(int n){
    int i, j;
    for(i=1; i<=n/3; i++)
        for(j=1; j<=n; j+=4)
            printf("Hello\n");
}
```

Scanned with CamScanner

Page 18
Date

- Understanding void pointer - def.
- Where void * can point
- Eg code (typecast)
- Can we dereference without typecasting
- Application of void pointer
- Syntax
- Where malloc/calloc used,
- What is a NULL pointer
- Eg code
- Use of NULL pointer - 1
- Code
- Good practice
- Use of NULL ptr - 2
- Code
- 2 things Malloc return
- Size of normal pointer - -
- Fact about null pointer - 1
- Eg code
- Fact 2 - size of null ptr
- Best practice - 2
- What is a dangling pointer
- Eg code - dangling
- - no more dangling
- Eg 2 of dangling pointer
- Why segmentation fault
- Wild pointer - other name
- Explain
- Eg code
- How to avoid wild pointer - 1

code
- 2
- code

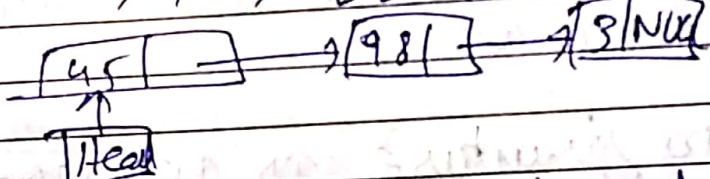
Scanned with CamScanner

Q. What is static memory allocation?
Q. Eg code
Q. Diagram for it.
Q. Dynamic memory allocation - def
Q. Diagram
Q. What is heap?
Q. Diff heap & stack
Q. Built in function
Q. Where code is stored?
Q. Where malloc declared (Library)
Q. malloc full form
Q. Syntax
Q. Usage
Q. Procedure
Q. Size t library
Q. Diagram of heap & void pointer
Q. Usage void pointer - 2
Q. How to use int pointer instead of void pointer
Q. Eg
Q. What to do
Q. Code / Diagram
Q. calloc() usage
Q. 2 diff b/w malloc
Q. Syntax calloc()
Q. Equivalent of 2 malloc
Q. What after malloc & calloc return null
Q. calloc full form
Q. malloc
Q. realloc
Q. " Library
Q. " Usage
Q. Syntax

- 2 things realloc return
- eg → 4 points
- Code
- where free used
- Syntax
- explain memory of heap
- who's responsibility
- eg
- Code
- Structure & Functions — 7 things (imp)
- Code for 1 call by value
- " " " " reference
- Structure variable as an argument — Roadmap
- Unlike arrays
- Code
- Pointer to structure as an argument — problem — better one
- Use of arrow operator
- eg Code
- Returning structure variable from a function
- Similar to —
- eg Code
- Returning a pointer to a structure from function
- eg of Passing array of structure as argument — Code
- eg Code
- Introduction to LL — target
- Limitations of arrays
- 3 types of linked list
- Navigation
- Single link — building block
- 2 parts of node

Scanned with CamScanner

Q. Represent a LL having data = 23, 54, 78, 90
Q. Array V/s LL (in terms of memory)
Q. Visualize array V/s LL
Q. 2 types of memory
Q. Who is sequential, linked
Q. Defn of self-referential structures
Q. Code
Q. Visualize
Q. Create a node of single LL
Q. In general "
Q. Why only 1 pointer
Q. Creating a node in C-code
Q. " " LL in " -code
Q. for




a) How do we add 2nd node
b) 1st way to add 3rd "
c) Why ineffective/disadvantages
d) 2nd way
Q. Traversing a single LL (counting nodes)
a) Code
b) Traversing meaning
c) Time complexity
d) Traversing a single LL (Printing the data)
a) Code
b) Time Complexity
Q. Traversing list (Array V/S LL)
Q. 4 Time complexities

Scanned with CamScanner

Page 122
Date

Insertion at end. LL
a) 1st way
b) Time complexity
c) 2nd way
d) Time complexity
Insertion at end array
a) case 1
b) Code
c) case 2
d) Code
e) Time complexities

Comparison LL v/s array → Time comp
why LL advantage over it
Inserting node at beginning of LL → Code → time
Common mistake - 2
Is there a simple way
Code
Time complexity
LL v/s array inserting data at beg
Code of array - not full
full
Time complexities
Inserting a node anywhere in LL - 3 steps
Code
array
visualize
steps to implement
Code
Compare time complexities
O(n) in array
what is n?

Scanned with CamScanner

Page
Date

Remember - 1
2
3
Deleting first node of single LL - Code
" " last " " - Note
- 2 steps
- Code
Why head pointer not returned
Can we use 1 pointer for deleting last node of list
Steps
Code
Time Complexity
Deleting at end - array
Time Complexity
Deletion at end (array V/S LL)
" " " beging (array V/S LL)
Array " " " code
Time complexity
Deleting node at particular position - Visualize
Eg
4 steps
Code
Why we passing & of head
Deleting whole LL, Can we directly delete head
So tell the steps
Code
Reverse a single LL - task
Visualize
Idea & Process
Iterative part
End part
temp →
temp2 →
Eg code

Scanned with CamScanner